

```
"""
```

```
scraper.py – pulls candidate events from  
tottenhamhotspurstadium.com.
```

```
IMPORTANT – read before running this on a  
schedule:
```

```
- Check
```

```
https://www.tottenhamhotspurstadium.com/robots.txt and the site's
```

```
Terms & Conditions before scraping  
regularly/automatically. This script  
fetches a small number of pages on demand  
(run manually or a few times  
a day), not a high-frequency crawl – keep  
it that way.
```

```
- The main /events listing renders its  
event grid with JavaScript, so a  
plain HTTP fetch (as done here) only sees  
the static navigation menu,  
not the full calendar. This scraper  
follows the "Concerts & Events" nav  
links to each individual event page (e.g.  
/events/1069172/nfl-2026),  
where the "Key Dates" section IS present  
in the static HTML.
```

```
- Football fixtures (Spurs home matches)  
are NOT on this events site at  
all – they live on tottenhamhotspur.com's  
fixtures pages. This scraper  
only covers concerts/NFL/other non-
```

football stadium events. Keep entering football fixtures manually (or via a football data API, see [prompt.md](#)) alongside this.

- Kick-off/start TIME is rarely given on these pages (only the date), so every scraped result is a **candidate** awaiting admin review – the admin must confirm/enter the correct start time before it reaches subscribers.

This is enforced by storing results in ``scraped_events``, separate from the live ``events`` table.

Usage:

```
python3 scraper.py          # scrape
and store candidates for review
"""

import re
from datetime import datetime

import requests
from bs4 import BeautifulSoup

import db

BASE_URL =
"https://www.tottenhamhotspurstadium.com"
EVENTS_PAGE = f"{BASE_URL}/events"
HEADERS = {"User-Agent": "Mozilla/5.0
(compatible; PersonalParkingAlertBot/0.1)"}

# Matches nav links like
/events/1069172/nfl-2026 (has a numeric
```

```

id),
# excludes generic pages like /events,
# /events/nfl-london-games (no id).
EVENT_LINK_RE = re.compile(r'href="
(/events/\d+/"')

# Matches "04 Oct", "11 October" style date
lines.
DATE_LINE_RE = re.compile(
    r'^\s*(\d{1,2})\s+([A-Za-z]{3,9})\.\.?
\s*$'
)

MONTH_LOOKUP = {
    m[:3].lower(): i for i, m in enumerate(
        ["", "Jan", "Feb", "Mar", "Apr",
        "May", "Jun",
        "Jul", "Aug", "Sep", "Oct", "Nov",
        "Dec"]
    ) if m
}

SKIP_LINE_PATTERNS = re.compile(
    r'^(on sale now!?|sold out|no
availability\.|coming soon|buy now)$',
re.I
)

def resolve_year(day: int, month: int,
today: datetime = None) -> int:
    """Event pages show day+month with no
year. Assume current year,
but roll forward to next year if that

```

```

date has already passed –
    stadium event pages don't advertise
past dates as upcoming."""
    today = today or datetime.now()
    candidate = datetime(today.year, month,
day)
    if candidate.date() < today.date():
        return today.year + 1
    return today.year

def fetch_event_links() -> list:
    """Get event page URLs from the static
nav menu on the events page."""
    resp = requests.get(EVENTS_PAGE,
headers=HEADERS, timeout=15)
    resp.raise_for_status()
    links =
set(EVENT_LINK_RE.findall(resp.text))
    return [BASE_URL + link for link in
links]

def parse_key_dates(html: str, source_url:
str) -> list:
    """Extract (event_date, event_name)
pairs from an event page's
'Key Dates' section using its text
structure, not brittle CSS
selectors (the site's markup may
change)."""
    soup = BeautifulSoup(html,
"html.parser")
    text = soup.get_text("\n")

```

```

    lines = [l.strip() for l in
text.split("\n") if l.strip()]

    results = []
    for i, line in enumerate(lines):
        m = DATE_LINE_RE.match(line)
        if not m:
            continue
        day = int(m.group(1))
        month_abbr = m.group(2)[:3].lower()
        if month_abbr not in MONTH_LOOKUP:
            continue
        month = MONTH_LOOKUP[month_abbr]

        # Look ahead a few lines for the
title, skipping status lines.
        title = None
        for lookahead in lines[i + 1: i +
4]:
            if
SKIP_LINE_PATTERNS.match(lookahead):
                continue
            if
DATE_LINE_RE.match(lookahead):
                break # hit the next date
block without finding a title
                title =
lookahead.lstrip("#").strip()
                break

        if title:
            year = resolve_year(day, month)
            event_date = datetime(year,
month, day).strftime("%Y-%m-%d")

```

```

        results.append((event_date,
title))

    return results

def scrape_events() -> int:
    """Scrape all linked event pages and
store new candidates for
    admin review. Returns the number of new
candidates found."""
    db.init_db()
    new_count = 0

    try:
        event_urls = fetch_event_links()
    except requests.RequestException as e:
        print(f"Failed to fetch events
page: {e}")
        return 0

    for url in event_urls:
        try:
            resp = requests.get(url,
headers=HEADERS, timeout=15)
            resp.raise_for_status()
        except requests.RequestException as
e:
            print(f"Failed to fetch {url}:
{e}")
            continue

        for event_date, title in
parse_key_dates(resp.text, url):

```

```
        before =
db.get_pending_scraped_events()

db.add_scraped_event(event_date, title,
url)

        after =
db.get_pending_scraped_events()
        if len(after) > len(before):
            new_count += 1
            print(f"New candidate:
{event_date} – {title}")

        return new_count

if __name__ == "__main__":
    count = scrape_events()
    print(f"\nDone. {count} new event
candidate(s) added – review them in the
Admin tab.")
```